

q-class

Q클래스 (Q-Type) 동작 원리

QueryDSL을 쓰면 컴파일 타임에 쿼리 오류를 잡을 수 있다.
그 핵심이 바로 **Q클래스**다. Q클래스가 어떻게 생성되고, 왜 필요한지 이해한다.

1. Q클래스란?

@Entity 가 붙은 JPA 엔티티 클래스를 기반으로 빌드 시점에 자동 생성되는 QueryDSL 전용 메타 클래스다.

엔티티 클래스	→ Q클래스 (자동 생성)
MemberInfoEntity.java	→ QMemberInfoEntity.java
OrganizationEntity.java	→ QOrganizationEntity.java
ScheduleEntity.java	→ QScheduleEntity.java

생성 위치: src/main/generated/

2. Q클래스가 생성되는 과정

1. 개발자가 @Entity 클래스를 작성한다
2. 빌드(gradle build)를 실행한다
3. QueryDSL의 APT(Annotation Processing Tool)가 동작한다
4. @Entity 붙은 클래스를 스캔한다
5. 각 필드를 QueryDSL 타입으로 매핑한 Q클래스를 생성한다

build.gradle.kts 설정

```
// QueryDSL APT 설정 (프로젝트에서 사용 중인 방식)
dependencies {
    implementation("com.querydsl:querydsl-jpa:5.x.x:jakarta")
    annotationProcessor("com.querydsl:querydsl-apt:5.x.x:jakarta")
    annotationProcessor("jakarta.persistence:jakarta.persistence-api")
}
```

annotationProcessor 가 빌드 시 APT를 실행시키는 핵심 설정이다.

3. 엔티티 필드 → Q클래스 필드 매핑

엔티티의 자바 타입이 QueryDSL 타입으로 1:1 변환된다.

```
// 엔티티 원본
@Entity
public class MemberInfoEntity {
    @Id
    private Long id; // → NumberPath<Long>
    private String mbName; // → StringPath
    private Boolean mbActivate; // → BooleanPath
    private LocalDateTime createdAt; // → DateTimePath<LocalDateTime>
    private MemberPositionType mbPosition; // →
EnumPath<MemberPositionType>

    @ManyToOne
    private OrganizationEntity organization; // → QOrganizationEntity (관계
필드)
}
```

```
// 자동 생성된 Q클래스 (src/main/generated/)
@Generated("com.querydsl.codegen.DefaultEntitySerializer")
public class QMemberInfoEntity extends EntityPathBase<MemberInfoEntity> {

    public final NumberPath<Long> id = createNumber("id", Long.class);
    public final StringPath mbName = createString("mbName");
    public final BooleanPath mbActivate = createBoolean("mbActivate");
    public final DateTimePath<LocalDateTime> createdAt =
createDateTime("createdAt", LocalDateTime.class);
    public final EnumPath<MemberPositionType> mbPosition =
createEnum("mbPosition", MemberPositionType.class);
    public final QOrganizationEntity organization;
}
```

타입 매핑 표

Java 타입	QueryDSL 타입	사용 가능 메서드
String	StringPath	.eq(), .contains(), .startsWith()
Long, Integer	NumberPath<T>	.eq(), .gt(), .goe(), .between()
Boolean	BooleanPath	.isTrue(), .isFalse()
LocalDateTime	DateTimePath<T>	.before(), .after(), .between()
Enum	EnumPath<T>	.eq(), .ne(), .in()
@ManyToOne 엔티티	Q엔티티타입	조인, 서브쿼리 등

4. 왜 컴파일 타임에 오류를 잡을 수 있는가

@Query (JPQL) - 런타임 오류

```
// 필드명 오타: mbName → mbNmae
@Query("SELECT m FROM MemberInfoEntity m WHERE m.mbNmae = :name")
List<MemberInfoEntity> findByName(@Param("name") String name);

// 빌드 성공 → 실행 시 에러 발생
// org.hibernate.query.sqm.PathElementNotFoundException
```

JPQL은 문자열이라 빌드 시점에 문법 검사가 안 된다.

실제 **API**를 호출해야 오류를 알 수 있다.

QueryDSL - 컴파일 오류

```
// 필드명 오타: mbName → mbNmae
queryFactory
    .selectFrom(qi)
    .where(qi.mbNmae.eq("홍길동")) // ← 컴파일 에러!
    .fetch();

// error: cannot find symbol
// symbol: variable mbNmae
```

Q클래스에는 `mbNmae` 라는 필드가 없기 때문에 빌드 자체가 실패한다.

IDE에서 빨간 줄이 뜨니까 코드 작성 중에 바로 잡을 수 있다.

필드명이 변경된 경우

1. MemberInfoEntity에서 `mbName` → `memberName`으로 필드명 변경
2. 빌드 실행 → Q클래스가 재생성됨 (`QMemberInfoEntity.memberName`)
3. 기존에 `qi.mbName`을 사용하던 코드 → 전부 컴파일 에러
4. 놓칠 수가 없다

이것이 QueryDSL의 타입 세이프(**Type-Safe**)가 의미하는 것이다.

5. Q클래스 사용법

기본 사용 - 정적 인스턴스

```
// 방법 1: 정적 인스턴스 (프로젝트에서 사용하는 방식)
private final QMemberInfoEntity qi = QMemberInfoEntity.memberInfoEntity;

// 방법 2: static import
import static
com.bunbo.organization.domain.member.entity.QMemberInfoEntity.memberInfoEn
tity;
```

별칭(alias)이 필요한 경우 - Self Join

같은 테이블을 두 번 조인해야 할 때 별칭으로 구분한다.

```
// 조직 self-join: 부서와 팀을 각각 참조해야 할 때
QOrganizationEntity qdept = new QOrganizationEntity("dept");
QOrganizationEntity qteam = new QOrganizationEntity("team");

queryFactory
    .select(qdept.orgName, qteam.orgName)
    .from(qdept)
    .leftJoin(qteam).on(qteam.parentId.eq(qdept.id))
    .fetch();
```

프로젝트 참고: MemberGroupRepoImpl.java 에서 이 패턴을 사용한다.

6. Q클래스 관련 트러블슈팅

Q클래스가 안 보일 때

원인: 빌드를 안 했거나, generated 폴더가 IDE에서 인식이 안 됨

해결:

1. ./gradlew clean build (Q클래스 재생성)
2. IDE에서 src/main/generated를 "Generated Sources Root"로 마크
3. IntelliJ: File → Invalidate Caches → Restart

엔티티 수정 후 Q클래스가 반영이 안 될 때

원인: 이전 Q클래스가 캐시되어 있음

해결:

1. src/main/generated/ 폴더 삭제
2. ./gradlew clean build
3. Q클래스가 최신 엔티티 기반으로 재생성됨

SQL → QueryDSL 사이에 Q클래스가 하는 역할 정리

SQL (문자열)	→ JPQL (문자열)	→ QueryDSL (자바 코드)
		↑ Q클래스가 여기서 동작 (필드를 자바 객체로 제
공)		
"mbName"	"m.mbName"	qi.mbName
↑ 오타 나도 런타임에 발견	↑ 오타 나도 런타임에 발견	↑ 오타 나면 컴파일에 발견 